

Innovation Document

Aiswaria Lajan, G00486594

This document describes the additional decisions and problem-solving carried out during the development of the Conference Management System, beyond what was strictly outlined in the project specification.

Database Connector Issue and Resolution:

During development, the standard mysql-connector-python package was installed as expected. However, when running the application on a Windows machine, the program would crash silently after the import statement with no error message shown, not even an exception. This made it very difficult to solve.

To isolate the issue, a separate diagnostic file test.py was created to test the database connection independently from the main application. This allowed the error to be identified step by step : the program printed 'start' and 'imported ok' but crashed on the connect line, which confirmed the issue was with the connector itself and not the application logic.

After further investigation, it was found that mysql-connector-python has a known compatibility issue on Windows where it calls a system exit instead of raising a Python exception when the connection fails. The solution was to switch to pymysql, which is a more lightweight and reliable MySQL connector for Python on Windows.

Additionally, the cryptography package was required because the version of MySQL on the ATU VM uses SHA256 password authentication, which pymysql does not support without it.

Packages used:

- pymysql
- cryptography

To install: pip install pymysql cryptography

Diagnostic Testing File (test.py):

A separate file called test.py was created during development to test and debug the MySQL and Neo4j connections independently. This file was not part of the specification but was a useful development tool that helped identify the connector issue described above.

Having a standalone test script is good development practice, it allowed individual components to be verified without running the full application, which made debugging much faster.

Cross-Database Validation in Options 4 and 5:

The specification noted that any attendee in the Neo4j database will always exist in the MySQL database, but it did not explicitly require checking MySQL before querying Neo4j. As a design decision, both options 4 and 5 were implemented to check MySQL first before interacting with Neo4j.

This approach has two benefits:

- It ensures the attendee exists before querying the graph database, which avoids unnecessary Neo4j connections.
- It allows the attendee's name to be retrieved from MySQL and displayed alongside the Neo4j results, giving the user more useful output.

Room Data Caching:

The specification required that rooms added to the database after option 6 is first used should not appear until the application is restarted. This was implemented using a global variable `_rooms_cache` which stores the result of the first database query.

On every subsequent call to option 6, the cached data is used instead of querying MySQL again. This is a simple form of application-level caching and also slightly improves performance since it avoids repeated database calls for data that does not change during a session.

Summary:

The main areas of additional work beyond the base specification were:

- Diagnosing and resolving a Windows-specific MySQL connector crash using a diagnostic test file and switching to pymysql
- Cross-referencing MySQL and Neo4j in options 4 and 5 for better validation and output
- Implementing session-level caching for room data